Universitatea Națională de Știință și Tehnologie Politehnica din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Detecting Colour in an Image

Proiect Python

Albu Iulia-Cosmina
Pitulice Laura-Teodora
423E

2024

1. Cerința

Acest proiect este o aplicație interactivă care permite utilizatorului să detecteze culorile dintr-o imagine printr-un click pe un pixel. Aplicația utilizează biblioteca Tkinter pentru interfața grafică și Pillow (PIL) pentru procesarea imaginilor.

2. Tehnologii utilizate

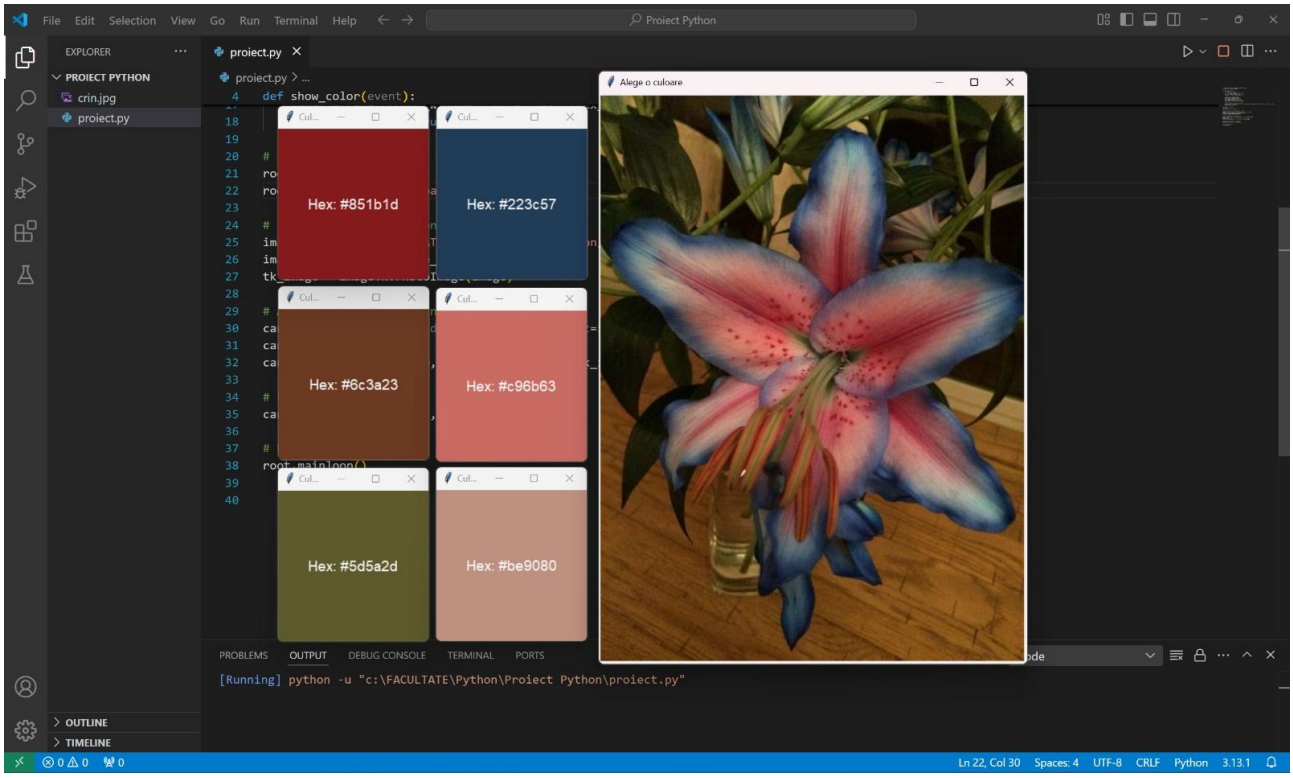
Pentru implementarea proiectului Detecting Colour in an Image, am utilizat Python datorită flexibilității, iar principalele tehnologii și biblioteci utilizate sunt:

I) Tkinter – Biblioteca standard Python pentru crearea interfețelor grafice. Am folosit Tkinter pentru a crea fereastra principală a aplicației, afișarea imaginii și gestionarea evenimentelor de click.

II) Pillow (PIL) – O bibliotecă Python utilizată pentru procesarea imaginilor. Aceasta ne-a permis să deschidem fișiere imagine, să le convertim în format RGB și să extragem culoarea pixelilor selectați.

3. Descrierea aplicației

3.1. Interfața



Aplicația Detecting Colour in an Image este un instrument interactiv care permite utilizatorului să identifice culorile dintr-o imagine cu un simplu click. Imaginea este afișată într-o fereastră principală, iar la click pe un pixel, aplicația detectează culoarea acelui pixel și deschide o fereastră suplimentară (pop-up). Fereastra afișează fundalul colorat cu nuanța selectată și codul său hexadecimal (HEX).

Această aplicație este utilă pentru designeri grafici, artiști sau dezvoltatori care au nevoie să extragă culori specifice dintr-o imagine, simplificând procesul de identificare a nuanțelor dorite.

3.2. Prezentare funcționalități

Aplicația Detecting Colour in an Image oferă o interfață simplă și intuitivă, iar funcționalitățile sale sunt implementate astfel:

A. Afișarea imaginii în aplicație
   ➔ Funcționalitate: Imaginea selectată este încărcată și afișată într-un widget Canvas, la dimensiunea sa originală.

```python

image_path = "C:/FACULTATE/Python/Proiect Python/crin.jpg"
image = Image.open(image_path).convert("RGB")
tk_image = ImageTk.PhotoImage(image)

canvas = Canvas(root, width=image.width, height=image.height)
canvas.pack()
canvas.create_image(0, 0, anchor="nw", image=tk_image)

```

Explicație cod:

- Imaginea este încărcată din calea specificată cu ajutorul Pillow și convertită în format RGB pentru a accesa culorile pixelilor.
- Este afișată pe un widget Canvas care are dimensiunile imaginii (**width=image.width, height=image.height**).

B. Detectarea culorii unui pixel la click
   ➔ Funcționalitate: Aplicația detectează coordonatele pixelului pe care utilizatorul a făcut click și extrage culoarea acelui pixel în format RGB, pe care o convertește în format HEX.

```python

def show_color(event):
    x, y = event.x, event.y
    r, g, b = image.getpixel((x, y))
    hex_color = f'#{r:02x}{g:02x}{b:02x}'

```

Explicație cod:

- La click pe imagine, coordonatele (x, y) sunt preluate din evenimentul **event**.
- Funcția **getpixel** din Pillow este utilizată pentru a extrage culoarea pixelului la acele coordonate.
- Culoarea este convertită într-un cod HEX utilizând formatarea stringurilor (**f'#{r:02x}{g:02x}{b:02x}'**).

C. Afișarea unei ferestre pop-up cu culoarea selectată

➔ Funcționalitate: După ce detectează culoarea, aplicația deschide o fereastră separată (pop-up) care are fundalul colorat corespunzător pixelului selectat și afișează codul HEX al culorii.

```
8
9   color_window = Toplevel(root)
10  color_window.geometry("200x200")
11  color_window.configure(bg=hex_color)
12  color_window.title("Culoare selectată")
13
14  label = Label(color_window, text=f"Hex: {hex_color}", bg=hex_color, fg="white", font=("Arial", 14))
15  label.pack(expand=True)
16
```

Explicație cod:

- Fereastra este creată folosind **Toplevel**, o funcție Tkinter dedicată ferestrelor secundare.
- Fundalul ferestrei este setat la culoarea selectată (**bg=hex_color**).
- Un **Label** afișează codul HEX, iar textul are un contrast ridicat (culoarea textului **fg** este **albă**).

D. Legarea clicului la funcționalitate

➔ Funcționalitate: Fiecare clic pe imagine declanșează funcția de detectare a culorii.

```
27
28  canvas.bind("<Button-1>", show_color)
29
```

Explicație cod:

- Evenimentul **<Button-1>** este asociat butonului stâng al mouse-ului.
- De fiecare dată când utilizatorul face clic pe imagine, funcția **show_color** este apelată pentru a detecta culoarea pixelului.

Concluzie:

Aplicația Detecting Colour in an Image este un instrument simplu, dar eficient, care demonstrează utilizarea Python pentru interacțiunea grafică și procesarea imaginilor. Prin integrarea bibliotecilor Tkinter și Pillow, aplicația oferă funcționalități intuitive, precum detectarea culorii unui pixel și afișarea codului HEX într-o interfață ușor de utilizat.

Cod:

```
from tkinter import Tk, Canvas, Toplevel, Label
from PIL import Image, ImageTk

def show_color(event):
    x, y = event.x, event.y
    # Obține culoarea pixelului din imagine
    r, g, b = image.getpixel((x, y))
    hex_color = f'#{r:02x}{g:02x}{b:02x}'

    # Creează o fereastră pop-up
    color_window = Toplevel(root)
    color_window.geometry("200x200")
    color_window.configure(bg=hex_color)
    color_window.title("Culoare selectată")

    # Adaugă textul cu hex-ul sub culoare
    label = Label(color_window, text=f"Hex: {hex_color}", bg=hex_color, fg="white", font=("Arial",
14))
    label.pack(expand=True)

# Fereastra principală
root = Tk()
root.title("Alege o culoare")

# Încarcă imaginea și convertește-o în RGB
image_path = "C:/FACULTATE/Python/Proiect Python/crin.jpg"
image = Image.open(image_path).convert("RGB")
tk_image = ImageTk.PhotoImage(image)
```

```
# Afișează imaginea pe un Canvas
canvas = Canvas(root, width=image.width, height=image.height)
canvas.pack()
canvas.create_image(0, 0, anchor="nw", image=tk_image)

# Eveniment pentru click pe imagine
canvas.bind("<Button-1>", show_color)

# Pornește aplicația
root.mainloop()
```